Technical Report: Exploring SocketCAN - Software-Only CAN/CAN FD Communication on Linux

Name: Sohan Satish Sutar BE170 | Course: Automotive Communication Networks (KPIT APEX Lab) | Problem Statement 1

1. SocketCAN overview

SocketCAN is the Linux kernel's CAN subsystem. It adds the `PF_CAN` socket family so CAN is used through the ordinary sockets API (`socket`, `bind`, `read`, `write`, `setsockopt`). A CAN controller, real or virtual, appears as a network interface (`can0`, `vcan0`) managed with `ip link`. Because applications talk to the kernel and not to a vendor library, the same code runs on any adapter, and on a purely virtual bus when no hardware exists. Each `CAN_RAW` socket has its own receive queue and its own kernel-level filter, so many applications can share one bus. This project uses the virtual interface `vcan0` (MTU 72, so CAN FD frames are possible) and Python 3's standard `socket` module, with no third-party packages.

2. Architecture used

```
Vehicle ECU --write--> [ Linux kernel PF_CAN / CAN_RAW ] <--> vcan0 (software bus)
                         | copy per matching socket |
                         v                          v
                  Dashboard ECU                Logger ECU
              (filter, decode, display,        (no filter, CSV file,
               watchdog)                        silence report)
```

Node	Role	Socket setup	Output
Vehicle ECU (`vehicle_ecu.py`)	Transmits Speed, RPM, Temperature from a drive-cycle model; optional CAN FD frame	`CAN_RAW`, `CAN_RAW_FD_FRAMES` with `--fd`	Frames on vcan0, 1 s status line
Dashboard ECU (`dashboard_ecu.py`)	Receives, decodes and displays; offline warning	`CAN_RAW_FILTER` for `--mode speed	rpm`	
Logger ECU (`logger_ecu.py`)	Records all traffic	No filter, FD enabled	`can_log.csv`

Design choice: receiving and drawing are decoupled. The Dashboard drains the socket with `select()` and redraws on a fixed timer, so heavy traffic does not freeze the display and a silent bus still triggers the watchdog.

3. CAN message definitions

Message	ID	Format	DLC	Encoding	Cycle
VehicleSpeed	0x100	Standard	2	uint16 little-endian, 0.1 km/h per bit, 0-120 km/h	100 ms

Message	ID	Format	DLC	Encoding	Cycle
EngineRPM	0x101	Standard	2	uint16 little-endian, 1 rpm per bit, 800-5000 rpm	50 ms
CoolantTemp	0x102	Standard	1	uint8, raw = °C + 40, 20-120 °C	500 ms
VehicleDataFD	0x200	CAN FD (BRS)	16 bytes	speed u16, rpm u16, temp u8, gear u8, odometer u32, 6 bytes reserved	100 ms
Unknown (test)	0x3A0	Standard	4	`DE AD BE EF`, deliberately undefined	manual

Example: 65 km/h = raw 650 = 0x028A, sent as bytes `8A 02`. Lower identifiers win arbitration, so Speed has the highest priority

4. Implementation notes

Vehicle ECU. Speed follows a random drive cycle with limited acceleration (6 km/h per second) and braking (10 km/h per second). A six-step gear model turns speed into RPM, with extra RPM under acceleration and a first-order lag for engine inertia. Coolant starts at 20 °C and warms toward about 88-108 °C with a 40 s time constant. All outputs are clamped to the required ranges. Each message has its own period timer, and the sender never bursts to catch up if it falls behind.

Dashboard ECU. Frames are decoded by identifier. Unknown identifiers are counted and optionally reported, never decoded. A watchdog records the arrival time of the Speed message (or any CAN FD frame).

Logger ECU. Each frame becomes one CSV row: timestamp, epoch, CAN ID, ID type, frame type, length, data. The file is flushed every 0.5 s. The Logger never transmits.

Testing. 19 offline unit tests cover encode/decode round trips, frame packing for classic, extended and FD frames, filter masks, model ranges over ten simulated minutes, CSV rows and dashboard behaviour.

5. Results of the learning challenges

Challenge 1 - Traffic observation. Yes, several applications observe the same traffic at once: the kernel copies every frame into the queue of each socket bound to vcan0, and one process reading a frame does not remove it for another. Communication continues unchanged while the Logger is active, because the Logger is a passive receiver. Delivery on vcan is in order and lossless at moderate rates; a sender does not see its own frames. A receiver that cannot keep up only loses frames in its own socket queue.

Challenge 2 - Message filtering. The Dashboard offers three modes, implemented with the kernel's `CAN_RAW_FILTER` option: all (no filter), speed (0x100) and rpm (0x101). A frame passes when `(received_id & mask) == (filter_id & mask)`. The mask also includes the extended-frame and RTR flags, so an extended frame whose low 11 bits equal 0x100 does not slip through.

Filtering happens before delivery, so the application is never woken for rejected frames. Observation: in speed mode only the Speed value updates and the other fields show filtered out; the Logger, with its own socket and no filter, still records everything.

Challenge 3 - Unknown message detection. A frame with ID 0x3A0 was injected with can_inject.py. The Logger records it because it accepts every identifier. The Dashboard does not show it as a signal: it increments the unknown counter and lists the ID, and prints an event line only with --show-unknown. The system keeps running, since CAN carries no notion of an unknown message at network level; meaning exists only in the application's message database. Robust receivers must therefore ignore, count or log unrecognised IDs and never crash or mis-decode them.

Challenge 4 - Transmission rate study. Expected behaviour: log size grows linearly with frame rate at roughly 70 bytes per CSV row (confirm with analyze_log.py), about 130 KB per minute at the baseline and about 13 MB per minute at 3000 frames per second. The Dashboard stays responsive because it redraws on a timer. At very high rates Python's CPU use rises and, if a receiver falls behind, the kernel drops frames silently for that socket. A virtual bus has no bandwidth limit, whereas a real 500 kbit/s classic CAN bus saturates at roughly 4000 8-byte frames per second, so vcan results show software limits, not bus limits.

Run	Rate (frames/s)	Mean period and jitter	Log growth (KiB/min)	Missing frames
Baseline 100/50/500 ms	32	100 / 50 / 500 ms nominal; jitter	about 125	vehicle “sent” total minus logger row count
10 ms each	300	10 ms nominal; jitter	About 1180	Same formula
1 ms each	upto 3000	1 ms nominal; jitter	Upto about 11800	Same formula

Conclusion to state: choose the lowest rate that meets the signal's dynamics, and keep logging efficient (binary format, rotation, filtering) at high rates.

Challenge 5 - Node failure study. After the Vehicle ECU is terminated, the Dashboard keeps displaying the last values, which would silently look valid without a diagnostic. The Logger sees no new rows and prints a bus-silent notice; SocketCAN raises no error, because on a bus a quiet sender is simply absent. An application detects loss of communication with a timeout watchdog on a periodic message, as implemented here, or with alive counters, heartbeat messages, end-to-end protection (counter plus CRC) and, on real hardware, bus-off and error-frame monitoring.

Challenge 6 - CAN FD exploration. See Section 6. The demo sends frame 0x200 with 16 bytes at the higher data bit rate (BRS).

Challenge 7 - Basic diagnostics. The Dashboard shows `WARNING: Vehicle ECU Offline` when no Speed message (0x100) or FD frame arrives within `--timeout` seconds (default 1.0 s, ten times the 100 ms period, so one or two lost frames never trigger it). It logs a 'back online' event when frames resume. In rpm filter mode the watchdog uses 0x101, because Speed is filtered out.

6. CAN FD observations

Property	Classic CAN	CAN FD
Maximum payload	8 bytes	64 bytes (DLC 9-15 map to 12, 16, 20, 24, 32, 48, 64)
Bit rate	One rate for the whole frame	Arbitration at nominal rate; data phase faster when BRS is set (commonly 2-8 Mbit/s)
New control bits	None	FDF (FD format), BRS, ESI
CRC	15 bit	17 bit (up to 16 data bytes) or 21 bit (longer)
Linux frame	`struct can_frame`, 16 bytes	`struct canfd_frame`, 72 bytes
Interface MTU	16	72
Remote frames	Yes	No

Application changes required. Set the interface MTU to 72; enable `CAN_RAW_FD_FRAMES` on the socket; send and receive 72-byte `canfd_frame` structures; validate that payload lengths are legal FD sizes; treat BRS and ESI as per-frame flags. Filtering code is unchanged. A socket that has not enabled FD only receives classic frames, so classic-only nodes simply never see the 0x200 frame.

Benefits. One 16-byte frame carries speed, RPM, temperature, gear and odometer that classic CAN needs four frames for, which cuts per-frame overhead; the faster data phase raises throughput; the larger payload suits diagnostics, security and software-update traffic; and the stronger CRC improves error detection. Mixed networks are possible, but a classic-only controller on a real FD bus will flag FD frames as errors, so migration needs planning.

7. Key learnings and conclusions

- SocketCAN turns CAN into a standard Linux network interface, so CAN software can be written, tested and demonstrated before hardware exists.
- Sockets are broadcast-friendly: adding a Logger or analyser costs nothing in the other nodes.
- Kernel filters reduce application load and keep receivers simple; identifier semantics live in the application's message database.
- Silence is not an error on CAN: reliable systems need explicit timeouts, alive counters or heartbeats.
- Unknown identifiers must be handled defensively (ignore, count, log).

- Virtual CAN validates logic and timing of software, not electrical behaviour: arbitration, bit timing, bus load and error handling still need real hardware testing.
- CAN FD needs small but specific application changes (MTU, socket option, 72-byte frames) and gives larger payloads and higher throughput.